Technical Review

Smart i-LAB Zone 5

Robust Desk/Zone 5 occupancy estimation through BSG DuckDB/Parquet ingestion, CV labels, power, mmWave, SEN55, and live-service gates

pctiope/smart-i-lab-testbed

May 2026

Technical Review Scope

What is reviewed

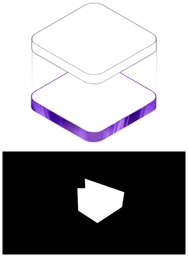
- A Zone 5-only pipeline for occupancy probability, live evidence, and model promotion.
- Source, label, feature, model, validation, performance, serving, and deployment contracts.
- Runtime boundaries between collectors, trainer, promoter, FastAPI app, and UI.

What is intentionally excluded

- Populated environment values, camera URLs, API keys, live logs, model binaries, and untracked artifacts.
- Runtime data/model artifacts under `data/`, `model/`, and `logs/`.
- Any claim that dashboard health alone proves a candidate model is promotable.

Source-grounded | No secrets | Production metrics | Operational gates

Problem Setting



Tracked dashboard and Desk 5 ROI assets.

Research setting

Desk/Zone 5 is a live lab zone, not a fixed benchmark. Occupancy evidence arrives through CV person counts, AIR-1 environmental readings, smart-plug power, mmWave state, SEN55 air-quality readings, and time.

- Streams can be stale, delayed, partial, or absent while the dashboard remains available.
- CV labels are proxy truth and inherit mask, lighting, and occlusion limits.
- The model must expose probability and readiness without learning missingness as occupancy.

Research Questions

1. Can Zone 5 combine environmental, power, mmWave, SEN55, CV, and time evidence without treating missingness itself as occupancy evidence?
2. Can short-history mmWave recency improve robustness while keeping `zone_occupied` CV-derived?
3. Can BSG DuckDB/Parquet ingestion feed the existing training and promotion contract without changing the serving pointer?
4. Can the same codebase support production user-systemd services and staged Compose checks without mixing ownership?

The answer is framed as a complete system walkthrough: data flow, package structure, model contract, training loop, serving endpoints, deployment boundaries, limitations, and source anchors.

System Scope And Ownership

Package scope

- Root BSG path owns Zone 5 ingestion and training input assembly.
- DuckDB state root: `data/smart_ilab.duckdb`.
- BSG support tables: `silver.zone5_sen55`, `silver.zone5_cv_labels`, and `silver.zone5_training_input`.
- Existing model artifacts and pointers remain in the Zone 5 package model root.
- Training target: `zone_occupied`.

Service ownership

- `run_bsg_live_collector.sh` owns live Bronze/Silver ingestion.
- Support-table seeding bridges CV and SEN55 rows into BSG.
- `run_bsg_trainer.sh` snapshots Silver training input.
- Promoter owns `model/production_run.txt`.
- FastAPI owns serving, readiness, and dashboard payloads.
- UI/Vite shell is separate from model validity.

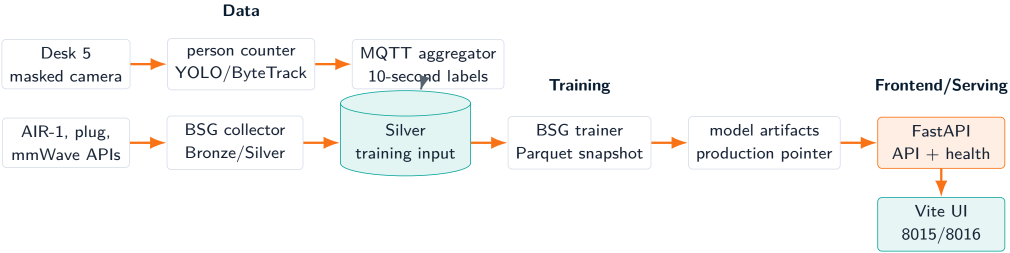
Boundary

Production uses user-systemd from `main`; Compose staging uses 8005/8016. Trainer and production-pointer policy stay production-owned.

Package Structure

Path	Review role
`api_ingestion.py`	Polls source APIs and writes Bronze/Silver BSG tables into DuckDB/Parquet state.
`bronze2silver_preprocess.py`, `silver2gold_preprocess.py`	Normalize source layers and keep the shared BSG layout consistent.
`zone5_training_migrated.py`	Builds `silver.zone5_training_input` from AIR-1, power, mmWave, SEN55, and CV support tables.
`seed_zone5_live_support_tables.py`	Seeds Zone 5 SEN55 and CV-label support tables and can rebuild the Silver training input.
`run_bsg_live_collector.sh`, `run_bsg_trainer.sh`	Production-facing wrappers used by user-systemd and operators.
`bsg_retrain_once.py`	Runs one BSG retrain cycle while preserving the artifact, status, and production-pointer contract.
`zone5_cv_time_features_package/zone5/feature_contract.py`	Declares model features, mmWave recency columns, target, and lookback choices.
`zone5_cv_time_features_package/web_app/main.py`	FastAPI routes, health payload, SSE stream, mask, and MJPEG endpoint.

End-To-End Runtime Flow



Data: collect + join | Training: snapshot + promote | Frontend: FastAPI + Vite

Architectural boundary

BSG owns Zone 5 data plumbing; training writes the existing model artifact and pointer contract; the Vite shell reads FastAPI and does not bypass promotion gates.

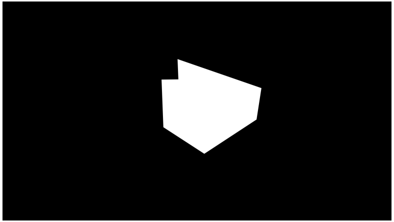
Active Services

Unit	Owned behavior
`zone5-person-counter.service`	Runs the masked Desk 5 person counter and emits count evidence plus latest annotated frame.
`zone5-mqtt-aggregator.service`	Converts person-count messages into 10-second CV buckets.
`zone5-sen55-collector.service`	Buckets SEN55 MQTT readings into package-local rows consumed by BSG support-table seeding.
`zone5-live-collector.service`	Runs `run_bsg_live_collector.sh` and keeps root DuckDB/Parquet BSG ingestion alive.
`zone5-live-app.service`	Serves FastAPI dashboard data, health, SSE, MJPEG, mask, and model probability when ready.
`zone5-trainer.timer/service`	Runs `run_bsg_trainer.sh`: seed support tables, snapshot Silver input, retrain, and promote if gated.
`zone5-vite-frontend.service`	Optional/persistent frontend shell; it does not decide model promotion.

Operational rule

Collection can continue, the app can stay available, and promotion can fail independently. The production pointer is the serving gate.

Person Counter Path



Tracked Desk 5 mask asset.

Source-grounded behavior

- Launcher path: `run_person_counter.sh`.
- Tracker script: packaged masked RTSP person tracker.
- Default tracker family: YOLO with ByteTrack-style tracking.
- Main artifacts: person-count CSV and latest annotated JPEG.
- MQTT count topic is evidence; it is not the final model target until bucketed.

Label chain

Person detections become count messages, then 10-second bucket rows, then the `zone_occupied` target.

CV MQTT Aggregation

Bucket contract

- Time base: 10-second sample boundary.
- Aggregation: median occupancy count per bucket.
- Late-grace handling allows delayed messages before flush.
- Output: package-local CV occupancy CSV.

Target rule

- Positive when median occupancy count reaches the configured threshold.
- Negative when median count is below threshold.
- Null when no CV label exists.
- Null CV buckets are not treated as vacancies.

Proxy truth | 10-second rows | Null-preserving

CV-derived target

Sensor Acquisition Paths

Family	Columns	Operational note
AIR-1 Zone 5	Temperature, humidity, CO2, and PM2.5	Core environmental inputs from `silver.air_1`.
Smart plug	`power_s5`	Core input from `silver.smart_plug_v2`; Zone 5 only.
mmWave	`mmwave_s5`	Core input from `silver.msr_2`; recency is derived later and target remains CV-derived.
SEN55	PM, temperature, humidity, VOC, NOx	Support-table input at `silver.zone5_sen55`.
CV labels	`zone_occupied`	Support-table labels at `silver.zone5_cv_labels`.
Time	Hour/day-of-week cycles	Deterministic cyclic features from timestamp.

No secret material

The deck names column contracts and source paths only. It does not include live API URLs, camera URLs, credentials, or generated logs.

BSG Live Collector Contract

Collector inputs

- Source API histories through `api_ingestion.py -all`.
- AIR-1, smart-plug, and mmWave device families.
- `SMART_ILAB_DUCKDB_PATH` for the shared DuckDB root.
- Polling and log paths from wrapper or user-systemd environment.

Collector outputs

- Bronze/Silver DuckDB and Parquet-backed state.
- Normalized `silver.air_1`, `silver.smart_plug_v2`, and `silver.msr_2` rows.
- Ingestion logs under the root runtime log path.
- No primary package-local joined training CSV for production.

The BSG collector refreshes source tables; the trainer builds and snapshots `silver.zone5_training_input` before fitting so appends do not mutate a run mid-fit.

BSG Training Support Tables

Support-table bridge

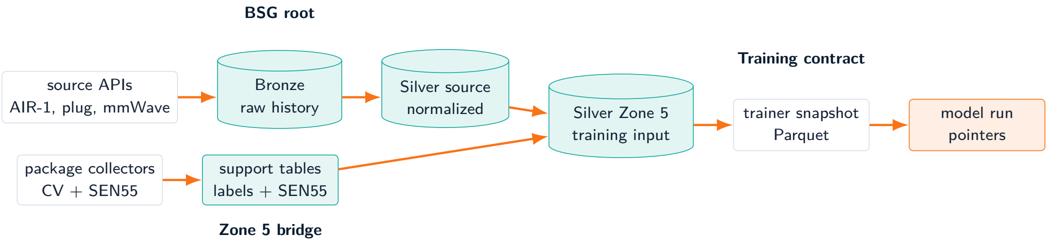
- `seed_zone5_live_support_tables.py` seeds `silver.zone5_sen55`.
- The same utility seeds `silver.zone5_cv_labels`.
- `-rebuild-training-input` rebuilds `silver.zone5_training_input`.
- The bridge is explicit because CV/SEN55 rows still originate from package-local collectors.

Trainer-facing output

- `zone5_training_migrated.py` joins Silver source and support tables.
- `zone_occupied` stays nullable and CV-derived.
- Missing indicators remain diagnostics/metadata, not model inputs.
- The resulting Silver table is the BSG trainer source.

Support tables | Silver input | CV target | No CSV-primary trainer

BSG Data Layer Chart



Bronze/Silver state | Explicit support bridge | Snapshot before fit | Existing pointer contract

Chart scope

This is a structural chart only: it names contracts and ownership boundaries without embedding live logs or untracked artifacts.

Feature Contract

Group	Columns	Model input?
AIR-1	`temp_s5`, `rh_s5`, `co2_s5`, `pm25_s5`	Yes
Power	`power_s5`	Yes
mmWave raw	`mmwave_s5`	Yes
mmWave recency	1m/3m/5m recent fractions, minutes since last occupied	Yes
SEN55	PM, temperature, humidity, VOC, NOx fields	Yes, optional family
Time	Hour and day-of-week sin/cos	Yes
Missing indicators	`*_missing` for raw feature columns	Diagnostics/metadata only
Target	`zone_occupied`	No, label

Current contract

`zone5_mmwave_recency_v1` extends the missingness-decoupled contract: raw sensors stay in the table, while missingness channels are not part of `FEATURE_COLUMNS`.

Core-Vs-Optional Gates

Core readiness families

- AIR-1 Zone 5 environmental fields.
- `power_s5`.
- `mmwave_s5`.
- Minimum present fractions are stored in artifact metadata and checked during training.

Optional SEN55 family

- SEN55 columns are preserved as raw sensors.
- Missing values can be neutral-filled for trainability.
- SEN55 dropout tests whether the model survives that family disappearing.
- SEN55 absence should not become an occupancy proxy.

Keep raw sensors | Gate core inputs | Do not learn missingness

mmWave Recency Without Leakage

raw `mmwave_s5` 10-second stream → past-only rolling builder → 1m, 3m, 5m occupied fractions → minutes since last occupied → Zone 5 input channels

- Recency is derived from samples at or before the current timestamp.
- Defaults protect rows with no prior mmWave occupancy.
- mmWave is an input feature and a health signal.
- `zone_occupied` remains the CV-derived target.

Training Table Semantics

One row means

- A floored 10-second timestamp for Zone 5.
- Latest joined sensor state available for that timestamp.
- CV label if the person-count bucket exists.
- Missing raw sensor values preserved before preprocessing.

One row does not mean

- A camera frame is guaranteed for every sensor row.
- Missing CV labels are negative labels.
- mmWave is target truth.
- A row is promotable evidence before split and coverage checks.

Review implication

The model claim is a table contract plus validation policy, not just a trained checkpoint.

Rolling Window Inputs

Window construction

- Candidate lookbacks: 15, 60, and 180 minutes.
- At 10-second cadence, these are 90, 360, and 1080 rows.
- Windows are built from feature rows only; labels supervise window endpoints.
- Scaling and fill values are learned from training partitions.

Tensor shape

Conceptually: feature channels by time. The CNN learns local temporal patterns before global pooling.

1D CNN Architecture

Stage	Source behavior
Input	`FEATURE_COLUMNS` channels across the selected lookback window.
Conv blocks	2 or 3 `Conv1d` blocks; base channels grow by block and cap at 256.
Kernel choices	First and late kernels are searched independently over compact odd sizes.
Normalization	Optional batch normalization after convolution.
Activation	Search over ReLU, GELU, and SiLU.
Pooling	Max-pooling after block 2 or after each late block.
Global pooling	Adaptive average or max pooling to one temporal summary.
Classifier	Dense layer, activation, dropout, final single-logit output.

TunableZoneOccupancyCNN Conv1d Adaptive pool Single binary logit

Hyperparameter And Training Loop

Search space

- Lookback, batch size, learning rate, weight decay.
- Number of convolution blocks, base channels, kernel sizes.
- Activation, normalization, pooling pattern, global pool.
- Dense units, dropout, scheduler, patience.

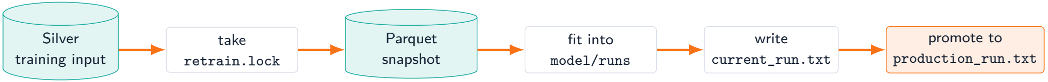
Training mechanics

- Optimizer: AdamW.
- Class imbalance: positive weight from train labels.
- Schedulers: none, cosine, or reduce-on-plateau.
- Objective: mean validation PR-AUC over eligible folds.
- Sampler: Optuna TPE with a fixed seed path.

Current production metrics included

This review includes the promoted 20260524T020721Z_2639c699 run, with detailed metrics tracked in `docs/performance/zone5-production-run-20260524T020721Z.md`.

Training Snapshot And Pointers



- The trainer snapshots `silver.zone5_training_input` under the model output root.
- The lock prevents overlapping retrains.
- `current_run.txt` is candidate state.
- `production_run.txt` remains the live-serving gate.

Validation Split Policy

Offline split discipline

- Latest calendar day is held out as blind-test evidence.
- Earlier eligible dates form one-day rolling validation folds.
- Strict mode requires usable train and validation labels.
- Bootstrap fallback is first-production recovery, not normal replacement.

Lookback viability checks

- A lookback must fit train and validation windows.
- Both-class windows are required unless explicitly in smoke/dev mode.
- Under-covered dates and rejected lookbacks are recorded in metadata.

Blind test | Rolling calendar | Both-class windows | Recorded split policy

Current Production AUCs

Promoted run

`20260524T020721Z_2639c699`: `zone5_mmwave_recency_v1`, 22 channels, 90-row / 15-minute lookback, 3 rolling-calendar CV folds, and a May 23, 2026 blind test.

Evaluation window	PR-AUC	ROC-AUC	Evidence note
Pre-test / training-window evaluation	0.8625	0.9839	40,062 windows before the blind-test day.
Blind test, May 23, 2026	0.7334	0.9230	8,277 windows held out from CV folds.
CV fold 1 validation	0.0799	0.0966	Validates May 20 after training on May 19.
CV fold 2 validation	0.8459	0.9676	Validates May 21 after training through May 20.
CV fold 3 validation	0.9364	0.9821	Validates May 22 after training through May 21.
Mean CV PR-AUC	0.6183 ± 0.3876	n/a	Aggregate stored for PR-AUC; mean ROC-AUC is not stored.

Source metrics JSON Manifest-verified No model artifacts tracked

Calibration And Class Balance

Window	Brier	BCE	Windows	Pos/neg windows	Pos. rate	Mean p
Pre-test	0.0419	0.1438	40,062	4,735 / 35,327	11.82%	0.1630
Blind test	0.0934	0.4475	8,277	1,806 / 6,471	21.82%	0.2814
CV fold 1	0.0800	1.2912	8,483	678 / 7,805	7.99%	0.0029
CV fold 2	0.0585	0.1998	8,551	1,643 / 6,908	19.21%	0.1734
CV fold 3	0.0683	0.2891	8,550	2,154 / 6,396	25.19%	0.3293

Split-level buckets/events

Split	Pos. buckets	Neg. buckets	Pos. events
Pre-test	4,735	35,533	207
Blind test	1,806	6,560	262

Confusion matrix limitation

The deployed artifact is threshold-free and no operating threshold is stored, so TP/FP/TN/FN are intentionally not reported.

Promotion Gates

Candidate quality gates

- Strict/allowed validation mode.
- Progressive strict-fold maturity.
- Blind-test PR-AUC and mean CV PR-AUC thresholds.
- Brier score and log-loss ceilings.
- Positive windows, positive buckets, and positive events.

Operational gates

- Smoke loading for artifact compatibility.
- Same-window non-regression against production when comparable.
- Atomic production pointer update on success.
- Failure returns status without stopping collectors or app serving.

Promotion Gate Funnel

candidate run snapshot → feature contract smoke load → quality gates threshold policy → same-window non-regression → atomic production pointer → FastAPI + Vite reload surface

Fail closed | Status recorded | Collectors continue | Serving pointer stays authoritative

Operational reading

The funnel separates candidate quality from serving safety: a failed promotion does not stop BSG collection, app health, or the previous production model.

BSG Smoke And Artifact Compatibility

What smoke checks protect

- `smoke_train_runtime.py` builds a runtime window from `silver.zone5_training_input`.
- Smoke runs check expected columns without depending on the old joined CSV.
- Optional output persistence exercises Silver and Gold training-output tables.
- Promotion smoke still verifies artifact and feature compatibility.

Why it matters

- Contract evolution is expected.
- A checkpoint alone is not enough to serve safely.
- Serving reloads should fail closed rather than silently mis-score inputs.
- The BSG smoke path validates data plumbing separately from model quality.

Review claim

Promotion is a software contract gate as much as a metric gate.

Serving Loop And Model Reload

Runtime inputs

- Production pointer path.
- BSG Silver source and training tables.
- Support CV and SEN55 tables seeded from package collectors.
- Configured tick interval, max gap, max age, and threshold.

Runtime outputs

- Latest occupancy probability event.
- History buffer for charting and SSE.
- Health payload with readiness and last-error context.
- MJPEG frame from live or file-backed grabber.

Pointer-polled History-sized Health-first Stream-capable

FastAPI Endpoint Surface

Endpoint	Purpose
`/`	Static dashboard shell.
`/api/health`	Readiness, data-source, model pointer, history, video, mask, and last-error state.
`/api/mask.png`	Packaged Desk 5 ROI mask when available.
`/api/current`	Latest inference/history event, or 503 when history is empty.
`/api/history?n=`	Bounded recent history for charts and diagnosis.
`/api/stream`	Server-sent events for live dashboard updates.
`/api/video.mjpg`	MJPEG video stream from RTSP or file-backed annotated frame source.

Interface constraint

The presentation change does not alter public API, schema, services, filenames, or Make targets.

Health And Readiness Semantics

Healthy enough to operate

- The app can answer `/api/health`.
- Video and mask routes can remain useful for diagnosis.
- Collectors can continue appending rows.
- Last error can explain an upstream or pointer blocker.

Not equivalent to

- A fresh production model exists.
- The candidate model passed promotion.
- All upstream sensors are fresh.
- The current dashboard probability is validated on new data.

Readiness is a composable health surface: serving status, source freshness, model pointer state, and error context are distinct signals.

Reproducibility And Deployment Boundary

Production user-systemd

Production is source-deployed from `main` and runs root BSG wrappers for live collection and training while preserving the package model/app contract.

- `zone5-live-collector.service` executes `run_bsg_live_collector.sh`.
- `zone5-trainer.service` executes `run_bsg_trainer.sh`.
- Runtime state stays out of git.

Docker Compose staging

`compose.zone5-bsg.yaml` validates BSG service wiring with the app on 8005 and Vite preview on 8016.

- Compose includes BSG ingestion, support-table, smoke, trainer, app, and frontend services.
- Compose does not add a production trainer timer.
- Manual proof precedes scheduled automation.

Operational Validity Checks

Code and config evidence

- Python compile checks for BSG modules, package, and web app.
- BSG migration and retrain wrapper tests.
- CV ground-truth unit tests.
- BSG Docker Compose config validation and image build.
- Runtime smoke from Silver training input.

Runtime evidence

- `/api/health` for readiness and last error.
- Timer/service status for collector and trainer ownership.
- Pointer files for candidate vs production model state.
- Logs inspected only locally; not embedded in the deck.

Why this belongs in research review

The model is only meaningful when data generation, validation, promotion, and live serving preserve the same contract.

Limitations And Threats To Validity

Data and label threats

- CV proxy truth can fail under occlusion, lighting shifts, or mask errors.
- Sensor/API outages can reduce feature coverage or delay joined rows.
- Class imbalance and sparse occupied windows can make metrics unstable.
- Delayed buckets can revise recent rows after first append.

Operational threats

- Upstream latency can stretch prediction cadence.
- Artifact compatibility must be explicit as contracts evolve.
- A healthy dashboard is not proof that a candidate is valid.
- Scheduled retraining must avoid overlapping live fits.

Roadmap

Near-term hardening

- Continue strict rolling-calendar promotion before expanding scheduled automation.
- Track model freshness, source freshness, and label coverage as first-class readiness signals.
- Keep mmWave recency monitored for leakage and stale-input behavior.

Research extensions

- Quantify feature-family ablations under the missingness-decoupled contract.
- Compare recency windows once enough positive evidence accumulates.
- Add reviewer-facing diagnostics that summarize split and coverage policy without exposing runtime secrets.

Source Anchors

Topic	Primary source paths
Pipeline map	`zone5_cv_time_features_package/PIPELINE.md`
BSG ingestion	`api_ingestion.py`, `bronze2silver_preprocess.py`, `silver2gold_preprocess.py`
BSG training input	`zone5_training_migrated.py`, `seed_zone5_live_support_tables.py`
BSG retrain wrapper	`run_bsg_trainer.sh`, `bsg_retrain_once.py`
Feature and target contract	`zone5_cv_time_features_package/zone5/feature_contract.py`
Model training	`zone5_cv_time_features_package/zone5/training.py`
Promotion	`zone5_cv_time_features_package/zone5/promote_model.py`
Current production metrics	`docs/performance/zone5-production-run-20260524T020721Z.md`
Serving endpoints	`zone5_cv_time_features_package/web_app/main.py`
Deployment boundary	`compose.zone5-bsg.yaml`, `zone5_cv_time_features_package/systemd/user/`

Closing point: Zone 5 treats model contract, labels, validation, and rollout gates as one technical system.